

Java Language – Complete Notes

Core Java concepts, syntax, OOP, collections, exceptions, files and modern Java basics.

1. Java Introduction

Key points

Java is a high-level, object-oriented, class-based language. It is designed to be platform independent through the JVM.

JDK/JRE/JVM

JDK = development tools + JRE. JRE = libraries + JVM. JVM executes Java bytecode.

Example

Source code (.java) → javac → bytecode (.class) → JVM.

2. Java Program Structure

Syntax

```
public class Main { public static void main(String[] args) { System.out.println("Hello"); } }
```

Example

main() is the entry point used to start a basic Java application.

3. Variables & Data Types

Key points

Primitive types: byte, short, int, long, float, double, char, boolean. Reference types include arrays, classes and interfaces.

Example

```
int age = 21; double marks = 85.5; char grade = 'A'; boolean pass = true;
```

4. Operators

Key points

Arithmetic: +, -, *, /, % ; Relational: <, >, ==, != ; Logical: &&, ||, ! ; Assignment: =, +=, -= ; Unary: ++, --.

Example

```
int a=10,b=3; a%b gives 1.
```

5. Input & Output

Key points

System.out.print/println displays output. Scanner can read keyboard input.

Example

```
Scanner sc = new Scanner(System.in); int n = sc.nextInt();
```

6. Conditional Statements

Key points

if, if-else, else-if and switch are used for decisions.

Example

```
if (marks >= 40) System.out.println("Pass"); else System.out.println("Fail");
```

7. Loops

Key points

for, while and do-while repeat statements. Use break to exit and continue to skip an iteration.

Example

```
for(int i=1;i<=5;i++) System.out.println(i);
```

8. Arrays

Key points

Array stores multiple values of the same type with indexed access starting at 0.

Example

```
int[] a={10,20,30}; a[0] is 10; a.length is 3.
```

9. Strings

Key points

String represents text. Common methods: length(), charAt(), equals(), substring(), toUpperCase(), toLowerCase().

Example

```
"Java".length() returns 4.
```

10. Methods

Key points

A method is a reusable block of code. It can take parameters and return a value.

Example

```
static int add(int a,int b){ return a+b; }
```

11. OOP Concepts

Key points

Four major pillars: Encapsulation, Inheritance, Polymorphism and Abstraction.

Example

A class bundles data and methods; private fields with public getters/setters demonstrate encapsulation.

12. Class & Object

Key points

Class is a blueprint; object is an instance of a class.

Example

```
class Car { String color; } Car c = new Car();
```

13. Constructor

Key points

Constructor initializes an object and has the same name as the class. It has no return type.

Example

```
Car(){ color="Red"; }
```

14. Encapsulation

Key points

Keep data private and provide controlled access through methods such as getters/setters.

Example

```
private int age; public int getAge(){ return age; }
```

15. Inheritance

Key points

A child class can inherit accessible members of a parent class using extends.

Example

```
class Dog extends Animal { }
```

16. Polymorphism

Key points

Compile-time polymorphism commonly uses method overloading; runtime polymorphism uses method overriding.

Example

add(int,int) and add(double,double) are overloaded methods.

17. Abstraction

Key points

Abstract classes and interfaces define required behavior while hiding implementation details.

Example

```
interface Payment { void pay(); }
```

18. Interfaces

Key points

An interface specifies a contract that classes implement using implements. A class can implement multiple interfaces.

Example

```
class Demo implements Runnable { public void run(){} }
```

19. Packages & Access Modifiers

Key points

Packages organize classes. Access modifiers: public, protected, default, private.

Example

```
import java.util.Scanner; imports Scanner from java.util.
```

20. Exception Handling

Key points

try, catch, finally, throw and throws help handle exceptional situations.

Example

```
try { int x=10/0; } catch(ArithmeticException e) { System.out.println("Cannot divide by zero"); }
```

21. Collections Framework

Key points

List: ArrayList, LinkedList. Set: HashSet, TreeSet. Map: HashMap, TreeMap. Collections provide flexible data structures.

Example

```
ArrayList list = new ArrayList<>(); list.add(10);
```

22. Generics

Key points

Generics provide compile-time type safety and reusable classes/methods.

Example

```
ArrayList names = new ArrayList<>();
```

23. File Handling

Key points

Java provides classes such as File, FileReader, FileWriter, BufferedReader and BufferedWriter for file operations.

Example

FileWriter can be used to write text into a file.

24. Multithreading

Key points

A thread is a lightweight unit of execution. Threads can be created using Thread or Runnable.

Example

```
class Task implements Runnable { public void run(){ } }
```

25. Java 8+ Important Features

Key points

Lambda expressions, functional interfaces, Stream API, Optional and default interface methods are important modern Java features.

Example

```
list.stream().filter(x -> x > 10).count();
```

Quick Revision Sheet

Topic	Quick revision
JDK/JRE/JVM	JDK develops; JRE runs; JVM executes bytecode
OOP	Encapsulation, Inheritance, Polymorphism, Abstraction
Array	Fixed-size indexed collection
String	Immutable text object
Exception	try-catch-finally; throw/throws
Collection	List, Set, Map
Interface	Contract implemented by classes
Overloading	Same method name, different parameter list
Overriding	Child provides its own implementation
Thread	Independent unit of execution